

Relatório de Chat Server

ChatServer.java

Classe ChatServer

closeConnection(SocketChannel sc, SelectionKey key)

— Envia mensagens através de `PrintWriter` (idealmente com *auto-flush* para evitar latência), encerra todos os recursos de forma ordenada e trata as exceções de modo a não as propagar. Este método garante a higiene do ciclo de vida da ligação, evitando descritores abertos.

groupMessage(SocketChannel sc, String roomName, String message)

— Gere o envio de mensagens para um grupo/sala, assegurando a escrita com `PrintWriter` (com *auto-flush*). Recorre a `synchronized` para prevenir condições de corrida em coleções/estruturas partilhadas, preservando a consistência do estado.

handleBye(SocketChannel sc)

— Emite a notificação de saída, desencadeia o fecho correto de recursos e captura eventuais exceções, assegurando que a sessão termina de forma limpa.

handleClientMessage(SocketChannel sc, String message)

— Processa a mensagem recebida no âmbito do protocolo de chat, aplicando a lógica necessária sem efeitos colaterais além do seu escopo.

handleJoin(SocketChannel sc, String message)

— Realiza o *parsing* do comando de entrada numa sala (extração de parâmetros) e atualiza o estado segundo as regras do protocolo, incluindo associações do utilizador à sala.

handleLeave(SocketChannel sc)

— Executa a lógica de saída de uma sala, ajustando o I/O da sessão quando necessário e limpando o estado associado para evitar referências órfãs.

handleMessage(SocketChannel sc, String message)

— Encaminha mensagens públicas respeitando o contexto (p. ex., sala) e o protocolo definido, servindo de ponto de passagem controlado.

handleNick(SocketChannel sc, String message)

— Interpreta e aplica a alteração de alcunha (nickname), validando o pedido e atualizando o mapeamento de utilizadores de forma consistente.

handlePriv(SocketChannel sc, String message)

— Processa mensagens privadas ponto-a-ponto, preparando os streams e interpretando o comando para entrega ao destinatário correto de forma confidencial.

main(String args[])

— Ponto de entrada: configura parâmetros, instancia componentes, cria o `ServerSocket` e entra no ciclo de `accept()` para aceitar novos clientes. Estabelece a escrita com `PrintWriter` (com *auto-flush*)

para reduzir a latência.

processInput(SocketChannel sc, ByteBuffer clientBuffer)

— Lê dados linha a linha, tipicamente num ciclo até ao fecho do canal; procede ao *parsing* e encaminha os comandos para os manipuladores adequados.

sendMessage(SocketChannel sc, String message)

— Emite a mensagem no canal associado ao cliente, utilizando `PrintWriter` com *auto-flush* para minimizar atrasos perceptíveis na entrega.

ChatClient.java

Classe ChatClient

new ActionListener()

— Assegura o fecho adequado de recursos associados a ações do utilizador e trata exceções de forma controlada, mantendo a aplicação estável.

ChatClient(String server, int port)

— Estabelece a ligação TCP ao servidor, inicializa os streams de I/O (com `PrintWriter` em *auto-flush*) e integra com a UI Swing; as atualizações visuais devem ocorrer no *Event Dispatch Thread*.

new WindowAdapter()

— Implementa o comportamento necessário em eventos de janela (p. ex., encerramento), garantindo limpeza e consistência do estado do cliente.

actionPerformed(ActionEvent e)

— Executa a ação correspondente ao evento, fechando recursos quando aplicável e tratando exceções para assegurar resiliência.

handleConnectionClosure()

— Centraliza a lógica de encerramento da ligação, libertando recursos e atualizando o estado para evitar fugas e inconsistências.

main(String[] args)

— Ponto de arranque: configura parâmetros, cria os componentes do cliente e estabelece o mecanismo de escrita com `PrintWriter` (*auto-flush*), promovendo uma comunicação responsiva.

newMessage(String message)

— Envia uma nova mensagem para o servidor com emissão imediata (*auto-flush*), garantindo que o utilizador observa resposta célere.

printMessage(final String message)

— Apresenta a mensagem recebida na interface, limitando-se à renderização para manter a separação de responsabilidades.

processServerMessage(String response)

— Realiza o *parsing* das respostas do servidor segundo o protocolo, desencadeando as ações subsequentes adequadas.

run()

— Executa num *thread* dedicado (Runnable), mantendo um ciclo de leitura linha a linha até ao término da ligação; isola operações de I/O para evitar bloqueios, trata exceções e assegura fecho ordenado.

windowOpened(WindowEvent e)

— Realiza a inicialização necessária quando a janela é aberta, garantindo consistência do estado inicial e prontidão da UI.

Em termos gerais, a solução privilegia a **responsabilidade única** de cada método, o fecho correto de recursos, o tratamento apropriado de exceções e a comunicação fluida (com *auto-flush*). Esta abordagem promove um funcionamento **conciso e objetivo**, reduz o risco de bloqueios e evita sockets indevidamente abertos.